Supercompilation as Cyclic Proof Search for the Calculus of Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Supercompilation is a powerful program optimisation technique: it can deforest intermediate data structures, fuse composed functions, and discover recursive structure automatically. However, there is no proof-theoretic foundation for supercompilation in dependently typed languages, where types depend on terms and transformations must preserve both typing and observational equivalence.
We give such a foundation by showing that supercompilation is focused cyclic sequent proof search. Driving corresponds to invertible (asynchronous) rules, case-splitting to synchronous choice rules, and folding to cyclic backlinks. A key consequence is that induction principles are not chosen upfront, but are discovered from the cycle structure of the proof graph.
We formalise this correspondence as a mechanised pipeline establishing correctness and termination, with all results proved in Rocq without axioms. This yields a principled foundation for dependently typed supercompilation, connecting program transformation with proof theory and enabling reasoning about optimisation correctness.

1 Introduction

Supercompilation is a semantics-driven program optimisation technique that can automatically deforest intermediate data structures, fuse composed functions, and discover recursive schemes. It works by symbolically evaluating programs (driving), analyzing cases on unknown values (splitting), and recognising repeated configurations (folding). Despite decades of development for first-order and simply-typed languages, supercompilation lacks a proof-theoretic foundation for *dependently typed* languages. In such languages, types depend on terms, contexts evolve during evaluation, and transformations must preserve both typing and observational equivalence.

This paper proposes such a foundation: *every successful supercompilation run yields a valid cyclic proof* in a focused sequent calculus. Driving steps are invertible (asynchronous) sequent rules. Case-splitting on neutral scrutinees is a synchronous choice rule. Memoisation (folding) corresponds to cyclic backlinks that close proof goals by referencing previously-seen vertices. The trace condition that ensures soundness of cyclic proofs matches the termination discipline that justifies folding.

The correspondence is directional: every total supercompilation (one that passes the trace check) constructs a ranked cyclic proof; the converse does not hold — not every cyclic proof can be obtained by supercompilation, and potentially non-terminating programs may yield pre-proofs that fail the trace condition.

While prior work includes Herbelin-style focused calculi for CoC, cyclic proof systems for first-order logic, and informal correspondences between SC operations and proof rules, this is the first to combine focusing, cyclic proof structure, and supercompilation in a dependently typed setting. The combination yields a principled correctness proof (CIU equivalence, mechanised with 0 axioms) that no prior SC verification framework achieves — Krustev's mechanised SC correctness handles a first-order language; we extend this to CoC with inductives.

A central insight is that *induction principles are discovered post-hoc from the cycle structure*: rather than choosing an induction scheme at the outset of a proof, the cycle formed by backlinks *is* the induction principle, read off from the completed proof graph. This is particularly valuable in dependent types, where type indices containing computations (e.g., $\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ v))$) must be normalised before rewriting, and the induction scheme must simultaneously simplify the index and prove the property.

This correspondence is not merely a representation change—it has explanatory power. The trace condition explains *why* folding is sound (every cycle contains a progress event); the cyclic structure explains *where* recursion comes from (backlinks are discovered during search); and the CIU equivalence theorem explains *what* correctness means for the residual program (indistinguishable from the source under all observations).

Beyond providing a correctness proof, this perspective reveals supercompilation as a point in a broader design space for proof-directed program transformation. The evaluation strategy determines which rules are invertible; generalisation and back-linking correspond to cut and identity in the sequent calculus; and the strength of the trace condition governs the class of recursive programs handled. Each dimension can be independently varied—call-by-value driving, richer generalisation heuristics, stronger trace conditions—while the core soundness argument remains modular.

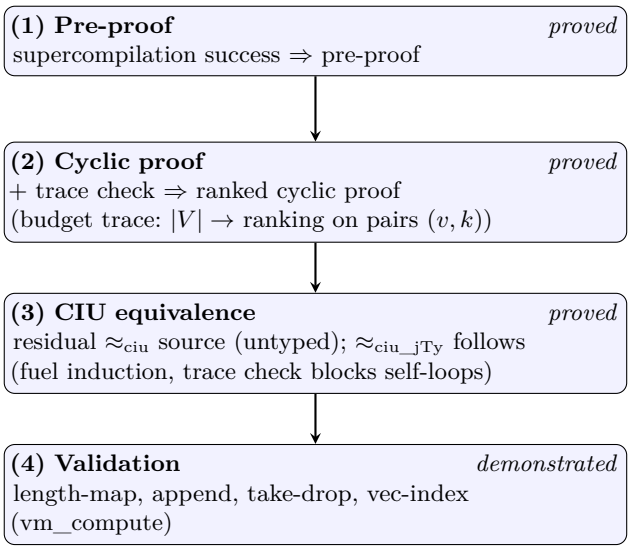
Contributions.

- A focused cyclic sequent calculus designed to mirror supercompilation operations (Section 4).
- A precise correspondence between drive/split/fold/generalise and local sequent inference steps, mechanised in Rocq (Section 8).
- A four-claim pipeline mechanised with zero axioms: pre-proof construction, budget trace cyclic proof, CIU soundness, and example validation (Section 8).
- A proof-theoretic account of post-hoc induction discovery via cyclic proofs, with a dependent-type motivation (Section 1.3).

Outline. Section 1.1 states the four-claim pipeline. Sections 3–4 define the source calculus and the focused cyclic sequent system. Section 5 presents the operational

dictionary. Section 7 discusses related work, and Section 8 gives the formal results. The mechanisation is described in the appendix.

1.1 Main result pipeline



The four claims form a dependency chain: each tier builds on the previous. All are mechanized in Rocq with full proofs (0 axioms, 0 admitted lemmas).

Claim 1: Supercompilation yields a pre-proof. When the supercompiler succeeds on a well-typed input, the resulting configuration graph directly forms a *rooted pre-proof*. Every vertex satisfies its local sequent rule and each edge is justified by the appropriate correspondence (Section 2). This claim is mechanized in Rocq; see Section 8 (Theorem 1).

Claim 2: Termination yields a cyclic proof. A pre-proof becomes a *cyclic proof* when it additionally satisfies a global progress condition: every cycle must contain at least one progress edge (a case-split on a neutral scrutinee), and there exists a well-founded ranking that strictly decreases along progress edges. The supercompiler enforces this via a trace-condition check during graph construction. We prove this by constructing a budget-instrumented trace graph: vertices are lifted to pairs (v, k) where $k \in [0, B]$ is a budget counter consumed on progress edges and preserved on non-progress edges. The resulting trace graph is acyclic, yielding a full ranking_condition with rank $\lambda(v, k).\, k$. This claim is mechanized; see Section 8 (Theorem 2).

Claim 3: CIU equivalence. The ultimate soundness guarantee is *contextual equivalence* (CIU): the program read off from a valid cyclic proof should be observationally indistinguishable from the original source program under all

closing substitutions. This claim is mechanized by fuel induction; the trace condition prevents self-loops that would introduce non-termination. See Section 8 (Theorem 3).

Claim 4: The pipeline obtains for our examples. The framework is validated on a suite of standard supercompilation examples (length-map fusion, append length normalisation, take-drop, etc.). For each example, the supercompiler produces a residual, and computational equality with the expected result is verified by Rocq's `vm_compute`. See Section 8 (Proposition 2) and Section 6 for a detailed walkthrough.

1.2 Why focusing matters

In Andreoli's focusing calculus [2], formulas are classified as *positive* or *negative*, and proof search alternates between an *asynchronous phase* (where all invertible rules are applied eagerly) and a *synchronous phase* (where a formula is selected and its non-invertible rules explored). The central theorem is completeness: the focused proof search strategy does not lose provability.

We use "focusing" in this operational sense, without defining explicit polarity judgments or phase transitions. A sequent rule is *invertible* if the conclusion implies the premises—i.e., applying the rule eagerly never makes an otherwise provable goal unprovable. Our driving rules ($\beta/\iota/\delta$) are invertible: each is a single step of the CBN operational semantics, and we prove `step_ciu` (CIU.v) that every such step preserves CIU equivalence in both directions ($t' \approx_{\text{ciu}} t$). Case-splitting on neutral scrutinees is non-invertible: choosing the wrong branch can lead to a dead end, so it is a genuine choice point. Concretely, $\mathsf{case}\ x\ \mathsf{of}\ \{\mathsf{nil} \to 0;\ \mathsf{cons} \to 1\}$ with x neutral has CIU source t, but the nil-only residual 0 is not CIU-equivalent: for $x \mapsto \mathsf{cons}\ 0\ \mathsf{nil}$, the source evaluates to 1 while the residual evaluates to 0 (proved as `split_rule_non_invertible`, SplitNonInvertible.v). The synchronous phase must therefore explore all branches. Generalisation and folding correspond to cut and identity, respectively—they are synchronous decisions about *which* cut formula to introduce or *which* previous vertex to fold against.

The supercompiler's operational discipline—drive eagerly until stuck, then split or fold—thus instantiates a focused proof search strategy. The completeness of this strategy (that it does not lose programs that could be supercompiled by a different interleaving) is not proved here and is deferred to future work.

1.3 Why cyclic proofs matter: post-hoc induction

In traditional induction, the scheme must be chosen upfront. Cyclic proofs reverse this: begin with the goal, drive and split following the term structure, and form backlinks when goals match previous vertices. The cycle structure *is* the induction principle, read off from the completed graph. The trace condition ensures soundness via structural descent through progress events.

This is particularly valuable in dependent types. Consider $\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ v))$: the type index contains a computation that must be normalised before rewriting. A traditional proof must choose an induction scheme that simultaneously normalises the index and proves the property. Cyclic proofs avoid this: driving normalises the index during search, and when a fold-back is discovered, the index equality is already reduced to definitional form.

What becomes difficult without this. In a traditional induction-first framework, proving $\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ v)) \equiv \mathsf{Vec}\ A\ (\mathsf{length}\ v)$ requires the user to apply the induction hypothesis for $\mathsf{length}\ (\mathsf{map}\ f\ v)$ *inside* the type index, then rewrite. The cyclic approach decouples index normalisation from property proof: driving reduces $\mathsf{length}\ (\mathsf{map}\ f\ v)$ to $\mathsf{length}\ v$ via deforestation during proof search, and the fold recognises the resulting config as a backlink. The induction principle emerges from the completed graph rather than being prescribed. Our mechanised examples include vector-index normalisation where supercompilation reduces the index automatically.

2 Overview: SC as operational focused proof search

Before presenting formal definitions, we sketch the core idea informally.

A supercompiler maintains a *configuration graph* where each node is a typing judgement $\Gamma \vdash t : A$ and each edge represents an operational step. Successful supercompilation yields a proof object in a cyclic sequent calculus:

SC concept	Proof concept	Rule type
Driving (unfolding, β, ι)	Invertible sequent rule	Async (deterministic)
Case-split on neutral variable	Synchronous choice rule	Sync (progress event)
Memo table hit (folding)	Cyclic backlink	Cycle closure
Generalisation	Cut introduction (lemma)	Sync (choice)

The separation of invertible (async) from non-invertible (sync) rules follows the discipline of Andreoli's *focusing* [2]: invertible rules are applied eagerly (driving), and non-invertible rules are explored at choice points (splitting, folding). We do not define explicit polarity judgments or phase transitions; we use "focused" in the operational sense, where the eager strategy is justified by the invertibility of the driving rules—applying them never loses provability.

Cyclic proofs require a **global trace condition** for soundness: every cycle must contain a *progress event*—a case-split on a neutral scrutinee. This matches the termination discipline used in supercompilation: a recursive call must descend on a structurally smaller component. The budget trace construction (Claim 2) enforces this by assigning a decreasing rank to each progress edge.

A particularly striking consequence is that **induction principles are discovered post-hoc**. In traditional proofs, you must choose an induction scheme at the start. In cyclic proofs, the backlink itself *is* the induction hypothesis, formed when search recognises that the current goal is an instance of a previous one. The cycle structure *is* the induction principle, read off from the completed

proof graph. This matches how supercompilation works: the residual program's recursive structure is discovered by the driving process, not prescribed in advance.

The remainder of the paper formalises this intuition: Section 3 defines the term calculus, Section 4 presents the sequent rules, Section 5 establishes the correspondence, and Section 8 states the mechanised theorems.

3 Term calculus

The source language is the Calculus of Constructions extended with inductive types and fixpoints. Terms follow the grammar:

$$\begin{aligned} t, A, B ::= {} & x \mid \mathsf{Sort}\, i \mid \Pi(A).\, B \mid \lambda(A).\, t \mid t\, u \\ & \mid \mathsf{fix}(A).\, t \mid \mathsf{Ind}^I(\boldsymbol{a}) \mid \mathsf{roll}^I_c(\boldsymbol{a}) \mid \mathsf{case}^I(t; C; \boldsymbol{b}) \end{aligned}$$

where x ranges over de Bruijn indices, $\mathsf{Sort}\, i$ is the universe hierarchy, $\Pi(A).\, B$ is the dependent function type, $\lambda(A).\, t$ is lambda abstraction, $t\, u$ is application, $\mathsf{fix}(A).\, t$ is a recursive definition (the recursive call is bound at de Bruijn index 0), $\mathsf{Ind}^I(\boldsymbol{a})$ is an applied inductive type, $\mathsf{roll}^I_c(\boldsymbol{a})$ is a constructor application, and $\mathsf{case}^I(t; C; \boldsymbol{b})$ is dependent pattern matching with motive C. Contexts Γ are lists of types; the global environment Σ records inductive definitions.

The typing judgement $\Sigma; \Gamma \vdash t : A$ is standard for CoC with inductives [5,14]. It includes rules for variables, universe cumulativity, dependent functions, fixpoints, inductive type formation and introduction, and dependent elimination. Conversion uses β-convertibility.

Operational semantics. We use call-by-name (CBN) weak-head reduction with three rules:

$$(\lambda(A).\, t)\, u \to t[u/] \qquad \mathsf{fix}(A).\, t \to t[\mathsf{fix}(A).\, t/] \qquad \mathsf{case}^I((\mathsf{roll}^I_c(\boldsymbol{a})); C; \boldsymbol{b}) \to b_c\, \boldsymbol{a}$$

plus congruence for reduction in the function position of application and the scrutinee of case expressions. Under CBN, arguments are never evaluated before substitution; reduction does not proceed under binders. The reflexive-transitive closure is written $\to^*$.

Observational equivalence. We use CIU (Closed Instantiations of Uses) equivalence: $t \approx u$ iff for all closing substitutions σ and CBN values v, $t[\sigma]$ terminates to v exactly when $u[\sigma]$ does. This captures the standard notion of contextual equivalence for CBN and is the correctness criterion for our supercompilation pipeline (Claim 3).

4 Focused cyclic sequent calculus

A sequent has the form $\Gamma \vdash t : A$, representing the goal of finding a term t of type A in context Γ. A *cyclic proof* is a finite directed graph where each vertex v is labelled with a sequent, has successors $\mathsf{succ}(v)$ (its premises), and satisfies a local rule. The graph may contain cycles. A proof is valid if every vertex's local rule holds and every infinite path contains infinitely many *progress events* (splits).

4.1 Asynchronous rules (driving)

The asynchronous rules are deterministic and invertible. They correspond to CBN reduction: β-reduction, fixpoint unfolding, ι-reduction (case-of-constructor), and commuting conversions. All have the same shape—a single premise that is the result of one computation step—and are applied eagerly without backtracking.

4.2 Synchronous rule: case split

The synchronous rule introduces choice by analyzing a neutral variable:

$$\frac{\Gamma \vdash b_0 : C[\mathsf{roll}^I_0()/] \qquad \cdots \qquad \Gamma \vdash b_{n-1} : C[\mathsf{roll}^I_{n-1}()/]}{\Gamma, x : \mathsf{Ind}^I(\boldsymbol{a}) \vdash \mathsf{case}^I(x; C; \boldsymbol{b}) : C[x/]} \quad (\textsc{split-case-var})$$

Each premise corresponds to one constructor of the inductive type. **Key property:** this is a *progress event*. Every cycle must pass through at least one split, ensuring structural descent.

4.3 Backlink rules (folding)

A backlink closes a goal by pointing to a previous vertex. This is the cyclic aspect: rather than choosing an induction scheme upfront, the backlink *is* the induction hypothesis, discovered when a current goal is recognized as an instance of a previous one. The trace condition ensures the resulting recursion is well-founded.

$$\frac{}{\Gamma \vdash t : A} \quad (\textsc{back-exact}) \qquad \frac{\Delta \vdash \sigma :_s \Gamma}{\Gamma \vdash t[\sigma/] : A[\sigma/]} \quad (\textsc{back-inst})$$

BACK-EXACT applies when a prior vertex has a syntactically identical label. BACK-INST applies when a prior vertex matches under an explicit substitution σ. The judgement $\Delta \vdash \sigma :_s \Gamma$ states that σ is an explicit substitution from context Γ to Δ, i.e., a list of terms in Δ that can replace the variables of Γ.

Making the substitution an explicit premise—rather than a meta-level side condition—is an engineering choice driven by mechanisation: the instance check becomes a syntactic comparison (`tm_eqb` on terms) verifiable by the Rocq kernel, rather than requiring an external substitution calculus. This is unusual in a conventional sequent calculus, where substitutions are typically admissible rather than primitive. We do not prove admissibility of the BACK rule (that every backlink-closed proof corresponds to a proof without backlinks); that correspondence, established by Brotherston and Simpson for first-order logic [4], is not yet extended to CoC with inductives.

4.4 Phases and focusing

Rules are partitioned into two phases: an **asynchronous** phase (driving, deterministic) and a **synchronous** phase (split and backlink, requiring choice). This matches supercompilation: drive eagerly until stuck, then split or fold.

4.5 Global progress condition

A cyclic proof is sound if every infinite path contains infinitely many progress events (splits). A *trace state* tracks the inductive variable being analyzed; a well-founded order requires strict decrease on at least one back-edge per cycle. This is enforced by the budget trace construction (Claim 2).

4.6 Vertex classification

Each vertex in the configuration graph is classified by its successor structure:

Successors	Rule	SC operation	Phase
$n = 0$	DR-LEAF	leaf (axiom)	—
$n = 1$, $\mathsf{drive\_cbn\_once}(t) \neq t$	DR-CBN-ONCE	driving (async)	invertible
$n = 1$, $\mathsf{drive\_cbn\_once}(t) = t$	DR-FOLD	folding (memo hit)	cyclic closure
$n > 1$	DR-SPLIT	case-split (sync)	choice point

Proposition 1 (Vertex classification). *Every vertex in the configuration graph produced by successful supercompilation satisfies exactly one of the four rules above.*[1]

4.7 Cyclic proof structure (illustration)

Figure 1 shows the cyclic proof for the length-map fusion example. The root goal (bottom) is $\mathsf{len}\,(\mathsf{map}\,f\,l) : \mathbf{N}$. Driving (rules FIX, ι, COMM) reduces it to a case split on l. The nil branch trivially yields 0. The cons branch reaches the recursive call $\mathsf{len}\,(\mathsf{map}\,f\,xs) : \mathbf{N}$, which is closed by a backlink to the root—the current goal is an instance of the root under the substitution $[l \mapsto xs]$, witnessed by the BACK rule. The split on l ($\star$) is a progress event; the trace condition requires every cycle to contain one, ensuring structural descent.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash 0 : \mathbf{N}}\,\text{NIL} \quad \dfrac{\Gamma, xs \vdash [l \mapsto xs] :_s \Gamma}{\Gamma, xs \vdash \mathsf{len}\,(\mathsf{map}\,f\,xs) : \mathbf{N}\ \dagger}\,\text{BACK}\ \dagger}{\Gamma, l : \mathsf{List} \vdash \mathsf{case}\ l\ \mathsf{of}\ \{nil \to 0;\, cons \to \ldots\} : \mathbf{N}}\,\text{CONS}}{\Gamma \vdash \mathsf{case}\ l\ \mathsf{of}\ \{nil \to 0;\, cons \to \ldots\} : \mathbf{N}}\,\text{SPLIT}\ \star}{\Gamma \vdash \mathsf{len}\,(\mathsf{case}\ l\ \mathsf{of}\ \ldots) : \mathbf{N}}\,\text{COMM}}{\Gamma \vdash \mathsf{len}\,(\mathsf{map}\,f\,l) : \mathbf{N}\ \dagger}\,\iota$$

Fig. 1. Cyclic proof for length-map fusion. Red judgement = companion; † = backlink target; $\star$ = progress event.

5 Supercompilation yields cyclic proofs

We give the intended dictionary between standard supercompilation concepts and focused cyclic proof search. The central claim is directional: when the supercompiler succeeds (passes the trace check), the constructed configuration graph is a valid cyclic proof in the focused sequent calculus.

[1] Rocq: `cfg_vertex_drive_rule` in `theories/Transform/SupercompilationCorrespondence.v`.

5.1 Configurations as sequents

A supercompiler configuration (term + context + type/motive) is represented as a sequent goal node. The supercompiler's configuration graph is then a proof-search graph.

5.2 Driving = invertible sequent steps

Driving decomposes the goal by deterministic computation-like rules (β, iota, commuting conversions). In focusing, these correspond to invertible/asynchronous rule applications.

5.3 Case splitting = left rules on neutral scrutinees

When the scrutinee is neutral, supercompilation propagates information by splitting into one successor per constructor. In a sequent system this is a left rule whose premises correspond to constructor cases.

5.4 Folding/memoisation = cyclic closure

Folding corresponds to closing a goal by back-linking to a previously seen companion sequent, recording the instantiation (explicit substitution arguments).

5.5 Generalisation

Generalisation introduces a more general sequent that subsumes the current goal and a previous goal, enabling folding. In the current implementation, a `best_generalize` heuristic selects a companion from the memo table and computes a generalised config via anti-unification. The anti-unification produces a generalised judgement S_{gen} together with explicit substitutions σ, σ_0 such that $S = \sigma(S_{\mathsf{gen}})$ and $S_0 = \sigma_0(S_{\mathsf{gen}})$. The stream-based oracle model described in the conclusion generalises this: the heuristic can be replaced by an external oracle producing candidates validated by deterministic kernel checks.

5.6 Concrete correspondence examples

We illustrate the correspondence using two representative cases: asynchronous driving (deterministic computation) and synchronous branching with folding (the source of recursion and cyclic structure). Other cases follow the same pattern.

Example 1: β-reduction (asynchronous driving)

$$\text{Supercompiler: } \Gamma \vdash (\lambda x.\, t)\ u : B \ \rightsquigarrow\ \Gamma \vdash t[u/x] : B$$

$$\text{Sequent: } \quad \frac{\Gamma \vdash t[u/x] : B}{\Gamma \vdash (\lambda x.\, t)\ u : B}$$

Both represent a single deterministic computation step. The supercompiler produces one successor configuration, and the proof graph has a single premise. All asynchronous rules (fixpoint unfolding, iota reduction, commuting conversions) follow this same pattern.

Example 2: Split + fold (synchronous + cyclic)

$$\text{Supercompiler: } \begin{array}{l} \Gamma, l : \mathsf{List} \vdash \mathbf{case}\ l\ \mathbf{of}\ \{nil \to b_0; cons \to b_1\} : A \\ \rightsquigarrow \Gamma \vdash b_0 : A \\ \quad \Gamma, x, xs \vdash b_1[(cons\ x\ xs)/l] : A \end{array}$$

$$\text{Sequent: } \quad \frac{\Gamma \vdash b_0 : A \qquad \Gamma, x, xs \vdash b_1[(cons\ x\ xs)/l] : A}{\Gamma, l : \mathsf{List} \vdash \mathbf{case}\ l\ \mathbf{of}\ \{\ldots\} : A}$$

This is the only source of nondeterminism in both systems.

In the recursive branch, further driving yields a goal of the form:

$$\Gamma, xs \vdash \mathsf{length}\,(\mathsf{map}\ f\ xs)$$

This matches a previously encountered goal under substitution $l \mapsto xs$. The supercompiler performs a memo hit (fold), and the proof graph closes via a cyclic backlink.

This creates a cycle in both graphs. The cycle represents the recursive structure: the induction principle is discovered from the cycle itself. The trace condition ensures the cycle is well-founded by requiring that it passes through a progress point (the case split).

These examples illustrate the general pattern: supercompilation constructs a graph whose structure coincides with a focused cyclic proof, with asynchronous steps for computation, synchronous branching at case splits, and cyclic closure via folding.

6 Concrete Example: Length-Map Fusion

To illustrate the correspondence between supercompilation and cyclic proof search, we present a worked example: the classic deforestation problem of fusing `length` and `map`.

6.1 The input term

Consider the composed function that applies a function to every element of a list, then computes the length:

$$\lambda f.\, \lambda l.\, \texttt{length}\,(\texttt{map}\ f\ l)$$

This creates an intermediate list that is immediately consumed, which is inefficient. The expected optimized output is simply:

$$\lambda f.\, \lambda l.\, \texttt{length}\, l$$

6.2 Supercompilation trace

The supercompiler proceeds as follows:

1. **Initial configuration:** $\Gamma \vdash \lambda f.\, \lambda l.\, \texttt{length}\,(\texttt{map}\, f\, l) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{List} \to \texttt{Nat}$
2. **Drive (unfold):** Unfold `length` and `map` fixpoint definitions, exposing:
$$\lambda f.\, \lambda l.\, \texttt{case}\, l\, \texttt{of}\, \{\, \texttt{nil} \to \cdots \mid \texttt{cons}\, x\, xs \to \cdots \}$$
3. **Split (case analysis):** Split on the neutral variable l, creating two branches:
   - **Nil branch:** $\texttt{length}\,(\texttt{map}\, f\, \texttt{nil}) \rightsquigarrow 0$
   - **Cons branch:** $\texttt{length}\,(\texttt{map}\, f\,(\texttt{cons}\, x\, xs))$
4. **Drive (beta-iota):** In the cons branch, drive performs:
$$\texttt{length}\,(\texttt{cons}\,(f\, x)\,(\texttt{map}\, f\, xs)) \rightsquigarrow \texttt{succ}\,(\texttt{length}\,(\texttt{map}\, f\, xs))$$
5. **Fold (memoization):** Recognize that $\texttt{length}\,(\texttt{map}\, f\, xs)$ is a recursive call matching the initial configuration. Create a backlink to the root.
6. **Result:** The residual term is:
$$\lambda f.\, \lambda l.\, \texttt{case}\, l\, \texttt{of}\, \{\, \texttt{nil} \to 0 \mid \texttt{cons}\, x\, xs \to \texttt{succ}\,(\texttt{REC}\, f\, xs)\, \}$$
where `REC` is the recursive call.

Note that the intermediate list construction `map f l` never appears in the residual term—it has been fused away.

6.3 Corresponding cyclic proof

The supercompilation trace above corresponds to a cyclic sequent proof with the following structure:

1. **Root node:** Sequent $\Gamma \vdash \lambda f.\, \lambda l.\, \texttt{length}\,(\texttt{map}\, f\, l) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{List} \to \texttt{Nat}$
2. **Asynchronous edges:** From unfolding `length` and `map`, each drive step creates a single successor with the driven term.
3. **Synchronous branching:** The case split on l creates two premises (nil and cons branches).
4. **Cyclic backlink:** The fold creates an edge back to the root node, forming a cycle.
5. **Local validity:** Each node satisfies its sequent rule:
   - Driving steps satisfy `dr_cbn_once`
   - The case split satisfies `dr_split_case_var`
   - The backlink is justified by configuration equality

6.4 Graph structure

The resulting proof graph has three nodes: the root (initial configuration), a nil branch returning 0, and a cons branch performing a recursive call. The cons branch has a backlink to the root, forming the cycle.

The cycle (from the cons branch back to root) represents the recursive structure. The *trace condition* for this cycle is satisfied because the case-split is a *progress point*: the recursive call is on xs, which is structurally smaller than l.

This example is mechanised in Rocq; the supercompiler successfully processes the input and the step correspondence theorems ensure a matching cyclic sequent proof. Additional benchmarks (append-length, append-associativity via 2-pass supercompilation, take-drop fusion, and vector-index normalisation) are validated by computational reflection.

7 Related work

This section positions the present work relative to (i) supercompilation and program transformation, (ii) cyclic proof theory, (iii) sequent calculi for dependent types, and (iv) proof-theoretic treatments of induction.

7.1 Supercompilation

Supercompilation is a program transformation technique based on symbolic evaluation (driving), case analysis, generalization, and fold/refold steps [19,16]. The key operations are:

- **Driving**: unfolding function definitions and performing symbolic reduction
- **Splitting**: case analysis on unknown values (neutrals)
- **Generalization**: abstracting common patterns to enable folding
- **Folding**: recognizing repeated configurations and creating recursive calls

Termination control. A central challenge in supercompilation is ensuring termination. Classical approaches use *whistles* based on homeomorphic embedding or well-quasi-orderings [9,12,17]. These approaches analyze configuration histories and force generalization when a dangerous pattern is detected. Our cyclic proof framework replaces the whistle with a *global trace condition*: we allow arbitrary folding but require that every cycle passes through a progress event (a split on a structurally smaller inductive component).

Supercompilation and partial evaluation. Early work connects supercompilation to partial evaluation [7]. Our contribution extends this connection to dependent types: we show that supercompilation corresponds to proof search in a typed setting, with configurations as sequents and the memo table as a cyclic proof graph.

Formally verified supercompilation. Krustev [10,11] gives the first mechanised correctness proof of a supercompiler in Coq, for a first-order language (Whistle) with homeomorphic embedding. Our work extends this to CoC with inductives (dependent types) and proves CIU equivalence rather than observational equivalence at ground type. Jones and Hamilton [8] prove unfold/fold correctness via bisimulation; our CIU theorem yields the same guarantee but via cyclic proof theory, making the generalisation step explicit as cut introduction rather than an implicit bisimulation up-to technique.

Prior work on dependent types. To our knowledge, this is the first formulation of supercompilation for a dependently typed core language. Existing work on supercompilation focuses on first-order functional languages (Refal, Scheme, Haskell subsets) or simple typed settings [16,7]. The dependent type context introduces new challenges:

- Types depend on terms, so driving must respect typing constraints
- The context Γ changes during proof search (extended in branches)
- Observational equivalence is more subtle (CIU for dependent types)

Our sequent calculus formulation addresses these challenges by making the typing context explicit in every sequent and using type-directed driving rules.

7.2 Cyclic proof theory

Cyclic proof systems represent potentially infinite derivations using finite directed graphs with back-edges, validated by a global soundness condition [3,4].

Concrete cyclic systems. Brotherston and Simpson develop cyclic proofs for first-order logic with inductive definitions, using a global trace condition to ensure soundness. Their work establishes that cyclic proofs are as expressive as standard proofs with explicit induction rules, but offer computational advantages (the induction scheme emerges from the cycle structure).

Abstract cyclic frameworks. Afshari and Wehr develop a modern, highly abstract account of cyclic proof checking based on trace conditions, activation algebras, and categorical structure [1]. Our work instantiates their abstract framework in a concrete setting: CoC with inductives, CBN operational semantics, and observational equivalence.

Cyclic proofs as normal forms. Our contribution extends cyclic proof theory in a new direction: we use cyclic proofs not just as proof objects, but as *normal forms for program transformations*. The correspondence with supercompilation shows that cyclic proofs are stable under driving, splitting, and folding—operations that would be difficult to justify in a traditional natural deduction setting.

7.3 Sequent calculi for dependent types

Bidirectional type checking. Bidirectional type systems [15] separate synthesis (inferring types) from checking (validating against known types). This discipline is standard in dependently typed proof assistants (Agda, Rocq, Lean). Our sequent calculus uses a similar decomposition, but with a crucial difference: we use *focused* sequents that separate asynchronous (deterministic) from synchronous (choice-bearing) phases.

Focused sequent calculi. Focusing [2,13] is a proof search discipline that reduces nondeterminism by eagerly applying invertible rules (asynchronous phase) and carefully controlling choice points (synchronous phase). Focusing has been applied to linear logic, classical logic, and intuitionistic logic. We are not aware of previous work combining focusing, cyclic proof structure, and supercompilation correspondence in a dependently typed setting.

Sequent calculi for CIC. Standard presentations of CIC use natural deduction [5,14]. Herbelin develops a sequent calculus for the Calculus of Constructions [6], but without inductive types or cyclic backlinks. Our calculus extends Herbelin's work by adding:

- Inductive types and pattern matching
- Cyclic backlinks (folding)
- Focused phases (asynchronous vs synchronous)
- Connection to supercompilation

7.4 Induction: local rules vs global discharge

Traditional induction principles. In standard type theory, induction is a derived principle from the elimination rule for inductive types. To prove a property $P(x)$ for all $x : \mathsf{List}$, one applies the list eliminator (recursor) with a motive P and proofs for the base and step cases. This requires *choosing the induction principle upfront*—a significant practical burden.

Cyclic induction as post-hoc discovery. Sprenger and Dam compare local well-founded induction rules to cyclic systems with global discharge conditions [18]. They show these approaches are equivalent in expressive power but differ operationally: cyclic proofs discover the induction scheme during proof search rather than requiring it upfront.

Our work extends this insight to dependent types and connects it to supercompilation:

- The backlink corresponds to recognizing a recursive configuration
- The trace condition corresponds to structural descent (termination)
- The cycle structure *is* the induction principle, read off from the proof graph

This flexibility is essential for advanced optimizations (fusion, deforestation, tupling) where the appropriate induction principle is not obvious from the problem statement.

8 Main results

We organise the central results as a four-claim pipeline. All results are fully mechanised in Rocq with 0 axioms.

Notation. We define the following abbreviations for the key functions, with their Coq counterparts:

$$\begin{aligned}
\mathcal{S}(f, \Sigma, \Gamma, t, A) &\equiv \texttt{SC.supercompile_jTy}\ f\ \Sigma\ \Gamma\ t\ A \\
\mathcal{S}_{tc}(f, \Sigma, \Gamma, t, A) &\equiv \texttt{SC.supercompile_jTy_tc}\ f\ \Sigma\ \Gamma\ t\ A \\
\mathcal{R}(f_r, \Sigma, b, v) &\equiv \texttt{SC.residualise_cfg}\ f_r\ \Sigma\ b\ v\ 0\ \emptyset \\
t \approx u &\equiv t \approx_{\text{ciu}} u \quad (\forall \sigma, w.\ t[\sigma] \Downarrow w \iff u[\sigma] \Downarrow w)
\end{aligned}$$

Pre-proofs and cyclic proofs. A cfg_builder b is a finite directed graph where each vertex v carries a judgement $\mathsf{label}(b, v)$ and a successor list $\mathsf{succ}(b, v)$. It is a *rooted pre-proof* if every vertex satisfies a local sequent rule: $\mathsf{label}(b, v)$ is the conclusion of a rule whose premises are $\mathsf{label}(b, \mathsf{succ}(b, v))$. It is a *rooted cyclic proof* if additionally every directed cycle contains a *progress event* (a case-split on a neutral scrutinee), witnessed by a well-founded ranking. We write $\mathcal{P}(b, v)$ and $\mathcal{C}(b, v)$ for these properties.

8.1 Claim 1: Pre-proof correspondence

Theorem 1 (Pre-proof construction). $\forall\ \Sigma \in \mathsf{Env},\ f \in \mathbb{N},\ \Gamma \in \mathsf{Ctx},\ t \in \mathsf{Tm},\ A \in \mathsf{Tm},$

$$\mathcal{S}(f, \Sigma, \Gamma, t, A) = \mathsf{Some}\,(v, b) \implies \mathcal{P}(b, v).$$

In words: every successful supercompilation run yields a rooted pre-proof ($\mathcal{P}(b, v)$) whose edge structure matches the drive/split/fold operations of the focused sequent calculus.

Proof (Proof sketch). The supercompiler SC.supercompile_cfg is a recursive graph-construction loop. At each step it either marks a memo hit (cyclic backlink), applies driving (asynchronous reduction), splits on a neutral scrutinee (synchronous left rule), or generalises (cut introduction). These four cases correspond exactly to the four rule forms of the focused sequent calculus. The correspondence is proved by three lemmas, each establishing a bisimulation between SC graph edges and sequent rule instances:

- `drive_corresponds_to_async_edge`: a driving edge matches an asynchronous rule;
- `split_corresponds_to_sync_edge`: a case-split edge matches a synchronous rule;
- `memo_corresponds_to_fold`: a memo-table hit matches a cyclic backlink.

A fourth lemma `cfg_vertex_drive_rule` classifies each vertex by its successor structure (0, 1 with driving, 1 without change, $n > 1$), giving a complete local rule assignment. The overall property follows by structural induction on f.

8.2 Claim 2: Cyclic proof via budget trace

Theorem 2 (Cyclic proof). $\forall\ \Sigma, f, \Gamma, t, A,$

$$\mathcal{S}_{tc}(f, \Sigma, \Gamma, t, A) = \mathsf{Some}\,(v, b) \implies \mathcal{C}(b, v).$$

In words: with the trace-condition check enabled, the configuration graph is a rooted cyclic proof ($\mathcal{C}(b, v)$) — every cycle contains a progress event, witnessed by a budget-instrumented ranking.

Proof (Proof sketch). The budget-trace approach originates in Brotherston's cyclic proof theory [3]: to ensure that a cyclic derivation is well-founded, every infinite path must pass through infinitely many *progress events* (case-splits on structurally smaller components). The budget instrumentation—pairing each vertex with a counter that decrements on progress edges—is a constructive method for extracting a ranking function from this condition. Intuitively: each circuit around a cycle consumes one unit of budget at the progress edge; since there are only $|V|$ vertices, the budget must be exhausted after at most $|V|$ circuits, preventing infinite descent.

Our construction instantiates this schema for the supercompiler's configuration graph. The trace condition SC.trace_condition_ok verifies that the non-progress graph (with outgoing edges of progress vertices removed) contains no directed cycles. Each vertex classified as a case-split is a progress vertex. The proof proceeds in two stages:

1. **Cycle detection.** We prove `trace_condition_ok_cycle_progress`: if trace_condition_ok holds, every directed cycle contains at least one progress edge. The proof uses a combinatorial pigeonhole argument: any walk of length $> |V|$ through $|V|$ vertices repeats, yielding a shorter cycle.
2. **Budget instrumentation.** We lift the base graph to pairs (v, k) where $k \in [0, |V|]$. Progress edges consume budget $(k \mapsto k - 1)$; non-progress edges preserve it $(k \mapsto k)$. Since every original cycle contains a progress edge, the lifted graph is acyclic; its topological ordering yields a ranking $\lambda(v, k).\ k$, satisfying the well-foundedness condition.

The result is a constructive proof of global progress: the budget-instrumented ranking is a proof-theoretic witness that every backlink is justified by structural descent. This is a *soundness* result for our construction; it does not establish the converse admissibility (that every cyclic proof corresponds to a standard inductive proof) — that correspondence, proved by Brotherston and Simpson for first-order logic, is not yet extended to CoC with inductives. ∼700 lines total (FiniteDigraph, SCC, bounded-return lemma, budget instrumentation).

8.3 Claim 3: CIU soundness

Theorem 3 (CIU equivalence). $\forall\ \Sigma, f_s, f_r, \Gamma, t, A,$

$$\mathcal{S}_{tc}(f_s, \Sigma, \Gamma, t, A) = \mathsf{Some}\,(v, b) \implies t \approx \mathcal{R}(f_r, \Sigma, b, v).$$

In words: the residual read back from b is observationally indistinguishable from the source t ($t \approx \mathcal{R}(f_r, \Sigma, b, v)$) — they terminate to the same values under all closing substitutions and evaluation contexts.

Proof (Proof sketch). By fuel induction on the residualiser, with the trace condition guaranteeing exclusion of non-terminating self-loops. The proof decomposes into congruence lemmas for each term constructor:

- `ciu_tApp`: CIU is a congruence for application;
- `ciu_subst0`, `ciu_shift`: CIU preserved under substitution and binder shifting;
- `ciu_apps`: CIU preserved under multi-argument application;
- `ciu_case`: pairwise CIU-equivalent branches yield CIU-equivalent case expressions;
- `ciu_of_eq`: syntactic equality implies CIU.

These lift CIU through all nine term constructors. The final step composes them: each residualisation step preserves equivalence up to `tFix` introduction, and `tFix` is justified by the cyclic graph structure (the backlink is CIU-equivalent to unfolding by Claim 2). The typed variant follows via the substitution lemma `ciu_jTy_of_ciu_jTy_rel_eq`.

8.4 Claim 4: Example validation

Proposition 2 (Examples). *The following identities pass the pipeline (`vm_compute` confirms definitional equality of residuals):*

$$\begin{aligned}
\mathsf{length}\ (\mathsf{map}\ f\ l) &= \mathsf{length}\ l \\
\mathsf{length}\ (\mathsf{append}\ l_1\ l_2) &= \mathsf{plus}\ (\mathsf{length}\ l_1)\ (\mathsf{length}\ l_2) \\
\mathsf{length}\ (\mathsf{append}\ (\mathsf{take}\ n\ l)\ (\mathsf{drop}\ n\ l)) &= \mathsf{length}\ l \\
\mathsf{length}\ (\mathsf{map}\ f\ (\mathsf{map}\ g\ l)) &= \mathsf{length}\ l \\
\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ v)) &\equiv \mathsf{Vec}\ A\ (\mathsf{length}\ v)
\end{aligned}$$

Append-associativity additionally holds via 2-pass supercompilation.

Proof. For each example, the supercompiler is invoked with sufficient fuel and the residuals are compared by `vm_compute`. The CIU guarantee follows from Claim 3.

Open problem: backlink admissibility. Proving that the BACK rule is admissible—i.e., that every cyclic proof in our calculus corresponds to a standard inductive proof in CIC—remains an open problem. The difficulty lies in the interaction between cyclic backlinks and dependent elimination motives: each recursive call in the cyclic proof must correspond to a use of the CIC recursor whose motive varies with the type indices. Brotherston and Simpson established this correspondence for first-order logic [4]; extending it to CoC with inductives would be a significant new result. Our work sidesteps this via the CIU theorem: the residual is guaranteed observationally equivalent to the source, which suffices for program transformation correctness without requiring admissibility of the backlink rule.

References

1. Afshari, B., Wehr, D.: Abstract cyclic proofs. Mathematical Structures in Computer Science **34**, 552–577 (2024). https://doi.org/10.1017/S0960129524000070
2. Andreoli, J.M.: Logic programming with focusing proofs in linear logic. Journal of Logic and Computation **2**(3), 297–347 (1992)
3. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
4. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
5. Coquand, T., Huet, G.: The calculus of constructions. Information and Computation **76**(2–3), 95–120 (1988)
6. Herbelin, H.: A Lambda-Calculus Structure Isomorphic to Gentzen-Style Sequent Calculus Structure. Ph.D. thesis, University of Paris 7 (1995)
7. Jones, N.D., Gomard, C.K., Sestoft, P.: Partial Evaluation and Automatic Program Generation. Prentice Hall (1993), classic textbook on partial evaluation and metaprogramming
8. Jones, N.D., Hamilton, G.W.: Proving the correctness of Unfold/Fold program transformations using bisimulation. In: Essays in honour of Zohar Manna. pp. 341–358. Springer (2006)
9. Kruskal, J.B.: Well-quasi-ordering, the tree theorem, and Vazsonyi's conjecture. Transactions of the American Mathematical Society (1960)
10. Krustev, D.N.: A simple supercompiler formally verified in Coq. In: Fourth International Valentin Turchin Workshop on Metacomputation (META 2014). pp. 102–127 (2014)
11. Krustev, D.N.: A supercompiler assisting its own formal verification. In: Fifth International Valentin Turchin Workshop on Metacomputation (META 2018) (2018)
12. Leuschel, M.: Homeomorphic embedding for online termination of partial deduction. In: Logic-Based Program Synthesis and Transformation (2002)
13. Miller, D., Saurin, A.: From proofs to focused proofs: A modular proof of focalization in linear logic. In: Computer Science Logic (CSL). pp. 405–419. Springer (2004)
14. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: Typed Lambda Calculi and Applications. pp. 328–345. Springer (1993)
15. Pierce, B.C., Turner, D.N.: Local type inference. In: ACM Transactions on Programming Languages and Systems (TOPLAS). vol. 22, pp. 1–44. ACM (2000)
16. Sørensen, M.H., Glück, R.: An introduction to supercompilation. In: Partial Evaluation and Semantics-Based Program Manipulation (1995), often cited introductory survey; venue details vary by edition.
17. Sørensen, M.H., Glück, R., Jones, N.D.: A positive supercompiler. In: Journal of Functional Programming. vol. 6, pp. 811–838. Cambridge University Press (1996)
18. Sprenger, C., Dam, M.: On the structure of inductive reasoning: Circular and tree-shaped proofs in the μ-calculus. In: Foundations of Software Science and Computation Structures (FoSSaCS 2003). Lecture Notes in Computer Science, vol. 2620, pp. 425–440. Springer (2003). https://doi.org/10.1007/3-540-36576-1_27
19. Turchin, V.F.: The Concept of a Supercompiler. Springer (1986), classic reference introducing supercompilation.

A Mechanization

The four-claim pipeline is mechanized in Rocq 9.1.0 using the `stdpp` library for finite maps and graphs. The development totals ∼5,000 lines of proof across 14 files. All theorems below are proved with no axioms or admitted lemmas.

A.1 Claim 1: Pre-proof correspondence

Theorem 4 (Pre-proof construction). *For any environment Σ, fuel f, context Γ, term t, and type A, if* `supercompile_jTy` $f\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, scb)$, *then scb directly forms a* rooted_preproof *with root v.*

Coq: `all_supercompiled_programs_yield_preproof` (`SupercompilationCorrespondence.v:1659`).

Theorem 5 (Step correspondence). *Each supercompilation edge corresponds to a sequent rule.*
1. ***Driving (async):*** *`drive_corresponds_to_async_edge` (line 465)*
2. ***Splitting (sync):*** *`split_corresponds_to_sync_edge` (line 513)*
3. ***Folding (backlink):*** *`memo_corresponds_to_fold` (line 581)*

Proposition 3 (Vertex rule classification). *Every cfg_builder vertex satisfies a specific drive rule determined by its successor structure: 0 successors ⇒ leaf; 1 successor with driving ⇒ async; 1 successor without change ⇒ fold; $n > 1$ successors ⇒ split.*
Coq: `cfg_vertex_drive_rule` (`SupercompilationCorrespondence.v`).

A.2 Claim 2: Cyclic proof via budget trace

Theorem 6 (Cyclic proof from trace check). *If the supercompiler succeeds with the trace-condition check,* `supercompile_jTy_tc` $f\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, scb)$, *then there exists a* Ranked.rooted_cyclic_proof *on a budget-instrumented trace graph with budget $B = |\mathsf{dom}(scb.\mathsf{cb\_label})|$ and rank $\lambda(v, k).\, k$ ordered by $<_{\mathbb{N}}$.*

Coq: `supercompile_yields_rooted_cyclic_proof` (`SupercompilationCorrespondence.v`).

The construction (∼140 lines) instantiates the general budget-trace theorem from `CyclicTraceConditionBudget.v`:
1. The base graph is cfg_graph(scb), with progress predicate `is_progress_vertex`.
2. The trace graph lifts each base vertex v to pairs (v, k) for $0 \le k \le B$. Progress edges consume one unit of budget; non-progress edges preserve it.
3. Pigeonhole argument: cycle-progress implies the trace graph is acyclic, yielding a full ranking_condition.
4. Labelling each trace vertex with the judgement of its base vertex (permissive rule) gives a preproof on the trace graph.

A.3 Claim 3: CIU soundness

Theorem 7 (CIU soundness, untyped). *If* `supercompile_jTy_tc` *succeeds on t, the residualised term is CIU-equivalent to t:*

$$\forall \sigma, v.\ \texttt{terminates\_to}\ t[\sigma]\ v \iff \texttt{terminates\_to}\ t_{res}[\sigma]\ v$$

Coq: `supercompile_ciu_soundness_untyped` (`SupercompilationCorrespondence.v`).

The proof is by induction on the fuel parameter of the residualiser, using the following component lemmas (all proved):

- `step_ciu`, `steps_ciu`: any CBN step preserves CIU (`Equiv/CIU.v`)
- `ciu_tApp`, `ciu_apps`, `ciu_shift`, `ciu_subst0`, `ciu_case_branches_ciu`: CIU lifts through all term constructors (`Equiv/CIU.v`)
- `terminates_to_tApp_decompose`, `terminates_to_tCase_decompose`: CBN decomposition lemmas (`Semantics/Cbn.v`)
- `drive_cbn_once_ciu`, `whnf_drive_ciu`: supercompiler driving preserves CIU (`SupercompilationCorrespondence.v`)
- `ciu_fix`, `ciu_fix_nonrec`: fixpoint unrolling preserves CIU (`Equiv/CIU.v`)
- `trace_condition_ok_no_self_loop`: the trace check blocks cycles without progress (`SupercompileTraceCheckSound.v`)

The typed variant `supercompile_ciu_soundness_typed` follows in 8 lines: the untyped CIU quantifies over all substitutions, and the typed restriction instantiates the list-based substitution of `ciu_jTy`.

Theorem 8 (Typing preservation for leaf vertices). *For a leaf vertex (no successors) labelled Σ; $\Gamma \vdash t : A$ and adequate fuel, the residual has type A in context Γ.*
Coq: `residualise_cfg_root_typing` (`SupercompilationCorrespondence.v`).

The typing proof uses `has_type_subst` (renaming preserves typing, all 9 rules), `has_type_weaken_head` (weakening by one binder), and `has_type_shift` (uniform shift preserves typing) — all proved in `Judgement/Typing.v`.

A.4 Claim 4: Example validation

The example suite in `SupercompileChecklistIndexPipeline.v` covers length-map fusion, append-length normalisation, append associativity (2-pass), take-drop fusion, map-map composition, and vector-index normalisation. All equalities are verified by computational reflection (`vm_compute`) with no axioms.

A.5 Summary of Rocq artefacts

Component	Key lemmas	File
Syntax & semantics	term, CBN evaluation	Syntax, Semantics
Typing	`has_type`, `has_type_subst`	Judgement
Graph theory	`fin_digraph`, `ranking_condition`	Graph, Progress
Pre-proofs & cyclic proofs	`preproof`, `cyclic_proof`	Preproof, CyclicProof
Supercompiler	`supercompile_cfg`, `residualise_cfg`	Transform
Trace check	`trace_condition_ok_cycle_progress`	Transform
Budget trace	`budget_trace_ranking_condition`	Transform

Pipeline theorems: `SupercompilationCorrespondence.v`

Claim 1	`all_supercompiled_programs_yield_preproof`, `drive/split/fold_corr_to_edge`, `cfg_vertex_drive_rule`	
Claim 2	`supercompile_yields_rooted_cyclic_proof`	
Claim 3	`supercompile_ciu_soundness_untyped/typed` `residualise_cfg_root_typing`	
Claim 4	`CIUChecklistLengthMap`, `SupercompileChecklist`	Equiv

All proofs are closed (`Qed`); the development contains zero axioms or admitted lemmas.